Developing a modular nf-core RNA-seq workflow

Wanchen Zhang and Kritika Manna

Computational Workflows course | 8 October 2026

Repository: https://github.com/wanchenzhang/rnaseq/releases/tag/v1.0.0

1 Introduction

Our project aimed to build a reproducible RNA-seq workflow using the nf-core template and reusable modules. The workflow processes RNA-derived sequencing reads to estimate gene expression across samples. It covers quality control, trimming, genome alignment, duplicate marking and gene-level quantification. Developing it required us to connect module inputs and outputs, choose suitable parameters and test different sample layouts.

Nextflow manages task dependencies and passes data between modules, while Docker provides the software environments. To make the analysis easier to repeat and reuse, we used public test data, validated input parameters, recorded software versions and documented the workflow on GitHub. The public GitHub repository supports findability and access, CSV/TSV formats support interoperability, and documented parameters, software versions and a tagged release support reuse. These choices support the FAIR principles [1, 2, 3].

We tested the workflow with five yeast samples, including single-end, paired-end and multiple-run inputs. We also compared Trim Galore with fastp and checked the resulting FASTQ files, TPM matrices and key outputs. This report describes our implementation, evaluates its functionality and performance, and discusses what is still needed for a more complete RNA-seq analysis [4].

2 Methods

2.1 Pipeline structure

The project was created using nf-core template 4.1.0. Table 1 lists the main files and their roles. The workflow uses reusable nf-core modules, with their connections defined separately from execution settings.

Table 1: Main files in the pipeline repository.

File or directory	Role
`main.nf`	Calls initialization, analysis and completion workflows
`workflows/rnaseq.nf`	Connects analysis modules and data channels
`nextflow.config`	Defines parameters, profiles and execution reports
`conf/modules.config`	Sets module arguments, resources and output paths
`conf/test.config`	Provides test inputs and resource limits
`nextflow_schema.json`	Defines parameter-validation rules
`assets/schema_input.json`	Defines samplesheet-validation rules
`modules/nf-core/`; `modules.json`	Store reusable modules and record their revisions
`README.md`	Documents usage, inputs, outputs and the workflow map
`bin/validate_outputs.py`	Checks published output files

Each samplesheet row contains a sample ID, an R1 file and an optional R2 file. Rows with the same sample ID represent technical runs of one sample. Initialization groups these rows, and CAT_FASTQ merges their files while keeping paired reads separate. Merged reads are passed to raw FastQC and the selected trimmer. Trimmed reads independently feed post-trimming FastQC and STAR (Figure 1).

2.2 Test data and computing environment

The integration test used the public nf-core samplesheet `rnaseq/samplesheet/v3.10/samplesheet_test.csv` and its associated test reference files: `genome.fasta`, `genes.gtf` and `transcriptome.fasta` [5]. The reads derive from yeast study GSE110004. Seven sequencing runs are grouped into five samples, covering single-end, paired-end and multiple-run inputs. Each run contains 50,000 reads or read pairs of approximately 101 bp. The small read subsets and reduced reference were used for workflow testing.

Computations ran locally in Ubuntu 26.04.1 LTS under Windows WSL2, using Docker 29.8.1. The environment had 16 logical CPUs, approximately 24 GiB configured WSL memory and 8 GiB swap. The test profile limited each task to four CPUs, 4 GB RAM and one hour. These are per-task limits, rather than a limit on the combined resources of the workflow.

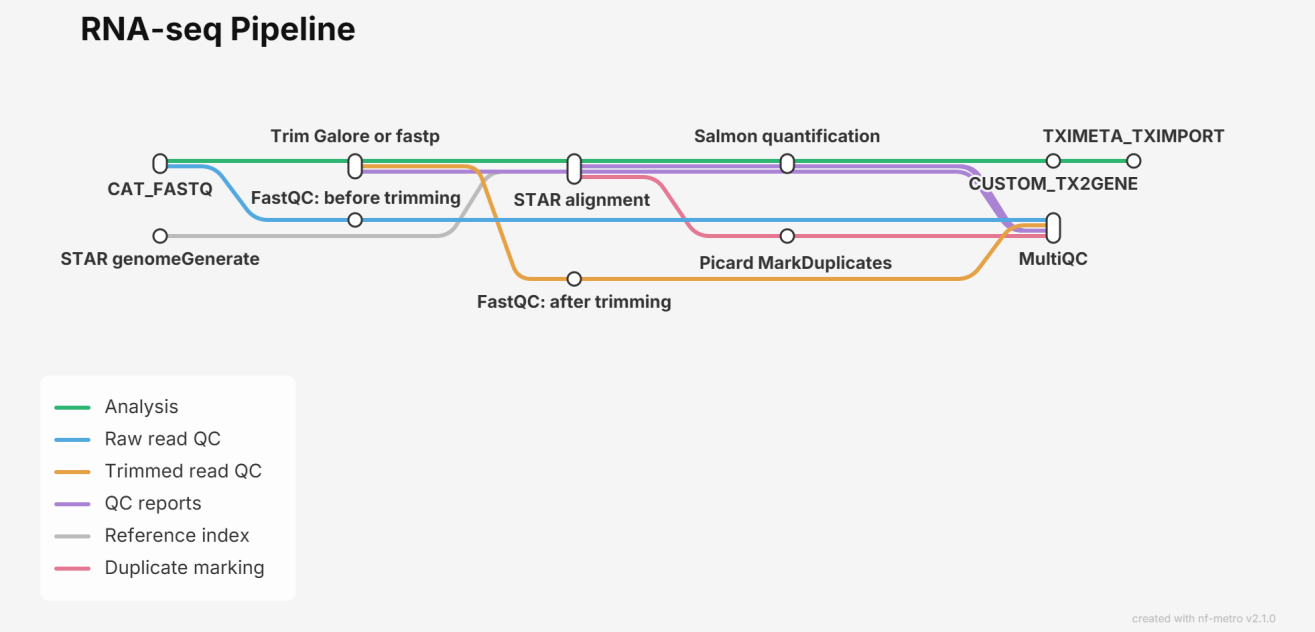


Figure 1: Implemented RNA-seq workflow and module dependencies.

Run records identified Nextflow 26.04.6 and nf-schema 2.5.1. Containerized tools included FastQC 0.12.1, Trim Galore 2.3.0, fastp 1.3.6, STAR 2.7.11b, Picard 3.5.0, Salmon 2.7.0, tximeta 1.20.1 and MultiQC 1.35. An earlier transcript-generation run used GFFREAD 0.12.7.

2.3 Preprocessing, alignment and quantification

Trim Galore is the default trimmer. The parameter `-trimmer fastp` selects fastp, while the `fastp_optional` profile also enables polyX trimming and overlap correction. PolyG handling can be set to `auto`, `on` or `off`. Overlap correction applies only to paired-end reads. Both branches use an independent FastQC process after trimming [11, 8].

STAR builds a genome index from FASTA and GTF and aligns the trimmed reads [6]. The settings `-quantMode TranscriptomeSAM` and `-outSAMtype BAM SortedByCoordinate` produce a transcriptome BAM and a coordinate-sorted genome BAM. Sorting is performed inside STAR with a sorting-memory allowance of 1,000,000,000 bytes. Index settings are `-genomeSAindexNbases 10` and `-sjdbOverhang 149`; these index parameters were inherited from earlier development. In particular, `-sjdbOverhang 149` was not adjusted to the approximately 101 bp test reads and should be reviewed before further analysis.

Picard marks duplicates in the sorted genome BAM and writes an index and duplication metrics. The setting `REMOVE_DUPLICATES=false` retains marked reads. Salmon independently quantifies STAR's transcriptome BAM using a matching transcript FASTA and GTF, with `-libType A` [7]. This alignment-based mode does not require a Salmon index. A supplied transcript FASTA bypasses GFFREAD; otherwise, GFFREAD generates transcript sequences from the genome FASTA and GTF [8].

CUSTOM_TX2GENE creates the transcript-to-gene mapping, and TXIMETA_TXIMPORT combines sample quantifications into gene and transcript tables, including a gene-by-sample TPM matrix [9]. MultiQC collects the QC reports, alignment logs, duplication metrics and quantification summaries [10]. Differential-expression analysis was outside the project scope.

2.4 Channel handling and validation

Channel operators adapt module interfaces and organize outputs. `map` transforms metadata/file tuples, `groupTuple` groups runs by sample, and `mix` combines report streams. `flatten().collect()` gathers files for MultiQC, while `toSortedList` orders quantification directories by sample ID before matrix generation. A value channel shares the reference index across samples.

Both trimming configurations were tested end to end using the test and Docker profiles. We ran `nf-core pipelines lint` to check the project against nf-core conventions. A separate Python validator checked FASTQ records, paired-read identities, read counts, TPM values and key output files. Performance measurements used the `realtime` field in Nextflow execution traces. Performance statistics were taken from one complete run per configuration, without repeated benchmark runs. No concurrency series or plain-Bash comparison was performed.

The tested configurations can be launched with:

```
nextflow run . -profile test,docker \
    --outdir results_test_trimgalore_only

nextflow run . -profile test,docker,fastp_optional \
    --outdir results_test_fastp_optional
```

2.5 Reproducibility and execution records

The tagged release provides a fixed version of the submitted workflow. The test profile defines the input files and execution settings, while Docker containers provide the software environments. Parameter files, software-version records and execution traces document how each analysis was run.

During development, `-resume` allowed completed tasks to be reused after changes. Cached tasks must be distinguished from newly executed tasks when comparing performance. The reported five-sample runs contained no cached tasks. Input reproducibility remains limited by the mutable test-data branch; pinning its revision and recording file checksums would make future runs easier to reproduce.

3 Results

3.1 End-to-end testing and output validation

The Trim Galore and optional-fastp configurations both completed the five-sample integration test. Each run recorded 39 completed tasks without cached tasks. The tests successfully processed single-end, paired-end and multiple-run samples and produced the expected outputs (Table 2). The supplied transcript FASTA bypassed GFFREAD in these runs; the transcript-generation route had been tested earlier. The `nf-core pipelines lint` checks also passed.

Table 2: Outputs confirmed in both five-sample test runs.

Stage	Confirmed outputs
Input preparation	Merged and trimmed FASTQ files
Quality control	Raw and trimmed FastQC reports, trimming reports and a combined MultiQC report
Alignment	Coordinate-sorted genome BAMs and transcriptome BAMs
Duplicate marking	Marked BAMs, indexes and duplication metrics
Quantification	Salmon estimates and a gene TPM matrix containing 124 unique gene rows and five sample columns
Execution records	Software versions, execution report, trace, timeline and DAG

The output validator returned `PASS=43 WARN=0 FAIL=0` for each run. FASTQ records and paired-read identities passed the checks, and trimming did not increase read counts. Both TPM matrices had the expected sample names, valid values and column sums close to one million.

Focused checks also confirmed that the validator rejected inconsistent sample layouts, duplicate runs, malformed FASTQ, mismatched mates and invalid TPM values. These checks verify specific output properties. BAM checks currently cover file existence and size, while original-to-merged read totals and binary integrity remain unverified.

3.2 Development problems and solutions

Testing exposed problems in module connections, resource configuration and parameter handling (Table 3). These failures helped us improve the workflow beyond the initial paired-end example.

3.3 Parallel execution and performance

The workflow provides opportunities for parallel execution across samples and independent analysis branches. Raw FastQC can run alongside trimming, and post-trimming FastQC can run alongside alignment. Picard and Salmon consume separate STAR outputs. In contrast, expression-matrix generation and MultiQC wait for their required sample results and reports. Actual concurrency depends on available resources and configured limits.

Figure 2 compares trimming-task execution time under the same test-profile resource limits. Trim Galore had a shorter recorded mean time than fastp with optional features. Performance statistics were taken from one complete run per configuration on small inputs, without repeated benchmark runs. This is therefore a preliminary tool comparison. We did not vary concurrency or compare with plain Bash, so no parallelization speedup was measured.

Table 3: Problems encountered during development and their solutions.

Problem	Solution	Lesson
Multiple single-end runs caused a Trim Galore symbolic-link error	Added CAT_FASTQ before trimming	Grouped sample inputs must match each module's expected file structure
STAR was terminated when WSL ran out of memory	Increased the WSL memory allocation	Available execution memory can differ from physical host RAM
Command-line Boolean values failed schema validation as strings	Set explicit Boolean values in a configuration profile	Parameter interfaces need testing as well as analysis modules

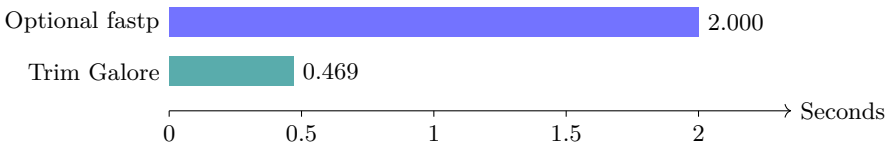


Figure 2: Mean trimming-task `realtime` across five samples, with one run per configuration. These measurements describe individual task execution, not whole-workflow runtime or parallelization speedup.

4 Discussion

4.1 Reproducibility and FAIRness

The workflow combines reusable module contracts, Docker environments, validated parameters, public inputs and traceable outputs. These mechanisms support nf-core's reproducibility goals [3], but using its template is not proof of community approval or full FAIR compliance. Our source is accessible on GitHub under the project's MIT license, and CSV/TSV outputs aid interoperability. The tagged GitHub release identifies the submitted code. Further improvements include archiving the release with a persistent identifier, pinning input and reference revisions, recording checksums and container digests, and providing richer sample metadata and an archived execution manifest. Container engine and Java versions should be captured per run rather than inferred from a later environment inspection.

4.2 Limitations and outlook

The small read subsets and reduced reference demonstrate workflow functionality rather than biological validity. The timing comparison also included optional fastp features, so it does not isolate differences between the tools themselves. Larger datasets, repeated runs and controlled concurrency settings are needed for a stronger performance evaluation.

The different checks provide different kinds of evidence. Lint checks assess project conventions, integration tests check whether connected modules complete on the selected inputs, and output validation checks specific file properties. Together, they support confidence in workflow functionality, but do not establish biological accuracy or portability to every execution environment. A useful next step is to run these checks automatically in continuous integration, so that future changes are tested against the same input cases.

The current workflow supports short-read bulk RNA-seq. Long-read and single-cell RNA-seq are possible future extensions (Figure 3). For long-read data, we could use platform-specific preprocessing, minimap2 for spliced alignment, and bambu for transcript discovery and quantification [12]. For common barcode/UMI-based single-cell data, STARsolo could perform alignment, barcode processing and UMI counting to produce a gene-by-cell count matrix. This would require the correct library layout, barcode whitelist and cell-level QC [13].

These branches have not been implemented. We still need to decide how to connect them to the current workflow and validate them with suitable datasets. Input checks and reporting could be shared, while preprocessing and quantification would use separate branches. Long-read and single-cell describe different aspects of the data; a long-read single-cell workflow would require further design.

Other priorities include BAM integrity checks, verification of read totals after merging, and clearer strandedness settings. A more complete bulk RNA-seq analysis would also include sample-level exploratory plots and differential expression analysis with biological replicates [4, 9].

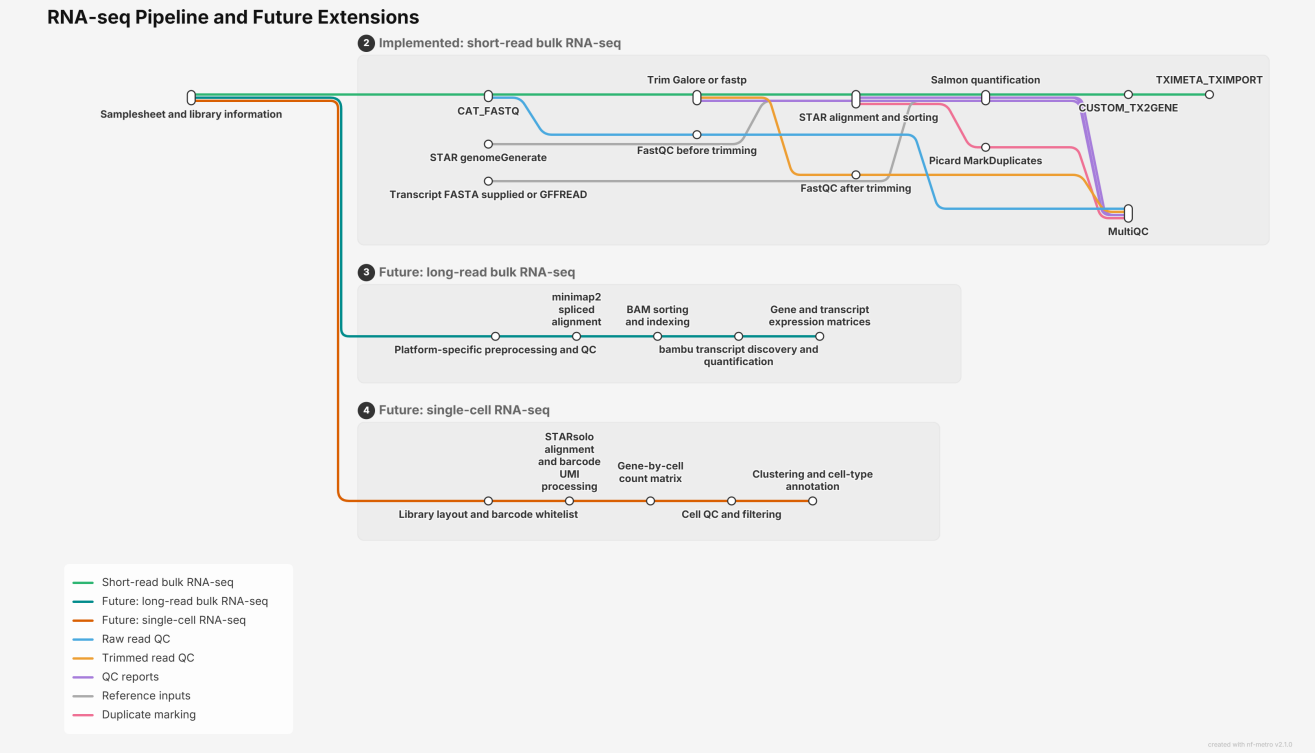


Figure 3: The implemented short-read bulk RNA-seq workflow and proposed long-read and single-cell extensions. The future branches have not yet been implemented or validated.

References

[1] Di Tommaso P et al. Nextflow enables reproducible computational workflows. *Nat Biotechnol.* 2017;35:316–319. doi:10.1038/nbt.3820.

[2] Wilkinson MD et al. The FAIR Guiding Principles for scientific data management and stewardship. *Sci Data.* 2016;3:160018. doi:10.1038/sdata.2016.18.

[3] Ewels PA et al. The nf-core framework for community-curated bioinformatics pipelines. *Nat Biotechnol.* 2020;38:276–278. doi:10.1038/s41587-020-0439-x.

[4] Conesa A et al. A survey of best practices for RNA-seq data analysis. *Genome Biol.* 2016;17:13. doi:10.1186/s13059-016-0881-8.

[5] nf-core. RNA-seq test datasets, `rnaseq` branch. https://github.com/nf-core/test-datasets/tree/rnaseq. Accessed 8 October 2026.

[6] Dobin A et al. STAR: ultrafast universal RNA-seq aligner. *Bioinformatics.* 2013;29:15–21. doi:10.1093/bioinformatics/bts635.

[7] Patro R et al. Salmon provides fast and bias-aware quantification of transcript expression. *Nat Methods.* 2017;14:417–419. doi:10.1038/nmeth.4197.

[8] Tool documentation: FastQC, https://www.bioinformatics.babraham.ac.uk/projects/fastqc/; GFFREAD, https://github.com/gpertea/gffread; Picard, https://broadinstitute.github.io/picard/. Accessed 8 October 2026.

[9] Soneson C, Love MI, Robinson MD. Differential analyses for RNA-seq: transcript-level estimates improve gene-level inferences. *F1000Research.* 2015;4:1521. doi:10.12688/f1000research.7563.1.

[10] Ewels P et al. MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics.* 2016;32:3047–3048. doi:10.1093/bioinformatics/btw354.

[11] Tool documentation: fastp, https://github.com/OpenGene/fastp; Trim Galore, https://github.com/FelixKrueger/TrimGalore. Accessed 8 October 2026.

[12] NIB-SI. longrnaseq: long-read RNA-seq pipeline. https://github.com/NIB-SI/longrnaseq. Accessed 8 October 2026.

[13] nf-core. scrnaseq pipeline, version 4.2.0. https://github.com/nf-core/scrnaseq/tree/4.2.0. Accessed 8 October 2026.